



Chef

pull based architecture

server agent

What is Chef



- Chef is an automation tool that provides a way to define infrastructure as code
- Infrastructure as code (IAC) simply means that managing infrastructure by writing code (Automating infrastructure) rather than using manual processes
- It can also be termed as programmable infrastructure
- Chef uses a pure-Ruby, domain-specific language (DSL) for writing system configurations

terraform → HCL

ansible → YAML

puppet → puppet language

chef → Ruby (DSL)

Why Chef ?



- Chef supports multiple platforms like AIX, RHEL/CentOS, FreeBSD, OS X, Solaris, Microsoft Windows and Ubuntu. Additional client platforms include Arch Linux, Debian and Fedora
- Chef can be integrated with cloud-based platforms such as Amazon EC2, Google Cloud Platform, OpenStack, Microsoft Azure and Rackspace to automatically provision and configure new machines
- Chef has an active, smart and fast growing community support
- Because of Chef's maturity and flexibility, it is being used by giants like Mozilla, Facebook etc.

Puppet vs Chef



■ Pros

- The complete User Interface → Enterprise (PE)
- Powerful reporting capabilities
- Grants access to a well-established support community

■ Cons

- Support for pre-Ruby versions is in the process of being scaled back
- Its model-driven approach equals less control when compared to code-driven approaches
- Advanced tasks require CLI, and since it is based on Ruby, you'll need to be familiar with the latter

■ Pros

- Its code-driven approach means greater flexibility and control of configurations
- The “Knife” tool lessens installation headaches
- Provides you with an extensive collection of configuration and module recipes

■ Cons

- Does not support push functionality
- If you don't already know Ruby and procedural coding, get ready for a steep learning curve
- It's a complex tool

Chef Installation and configuration



- Use Centos instance for chef installation

- > wget https://omnitruck.chef.io/install.sh

- > sudo bash install.sh -P chefdk -c stable



Building Blocks

Chef Resources



- A resource is a statement of configuration policy that:
 - Describes the desired state for a configuration item
 - Declares the steps needed to bring that item to the desired state
 - Specifies a resource type—such as package, template, or service
 - Lists additional details (also known as resource properties), as necessary
 - Are grouped into recipes, which describe working configurations

Recipe
↓
resources
↓
resource properties

Resource Syntax



- A resource is a Ruby block with four components
 - a type
 - a name
 - one (or more) properties (with values)
 - one (or more) actions
- The syntax for a resource is like this

```
type 'name' do
  attribute 'value'
  action :type_of_action
end
```

resource type resource name

config {
↓
property-value

```
package 'tar' do
  version '1.16.1'
  action :install
end
```

desired state

Chef client

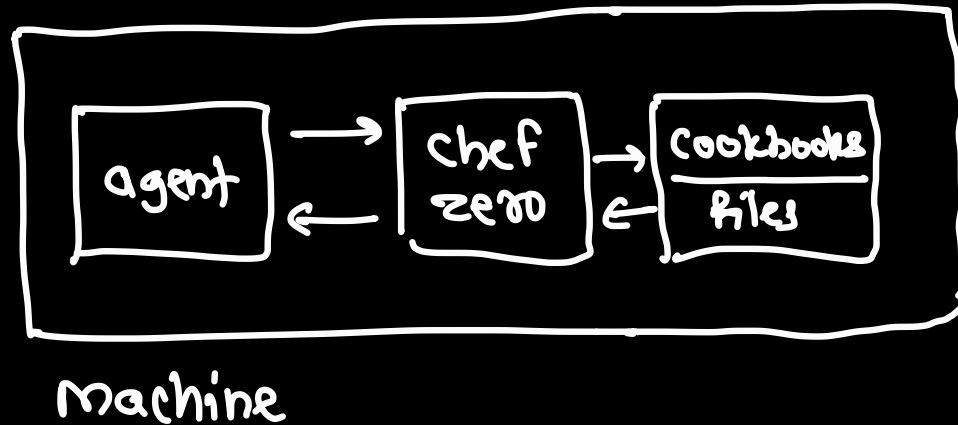


- Chef Infra Client is an agent that runs locally on every node that is under management by Chef Infra Server
- When Chef Infra Client runs, it performs all of the steps required for bringing a node into the expected state, including:
 - Registering and authenticating the node with Chef Infra Server
 - Building the node object
 - Synchronizing cookbooks
 - Compiling the resource collection by loading each of the required cookbooks, including recipes, attributes, and all other dependencies
 - Taking the appropriate and required actions to configure the node
 - Looking for exceptions and notifications, handling each as required
- The Chef Infra Client executable can be run as a daemon

Local Mode



- Local mode is a way to run the Chef Infra Client against the chef-repo on a local machine as if it were running against the Chef Infra Server
- Local mode relies on chef-zero, which acts as a very lightweight instance of the Chef Infra Server
- Local mode does not require a configuration file, instead it will look for a directory named /cookbooks
- If the client.rb file is not found and no configuration file is specified, local mode will search for a config.rb file
- Local mode will store temporary and cache files under the <chef_repo_path>/ .cache directory by default. This allows a normal user to run the Chef Infra Client in local mode without requiring root access.



Chef Zero

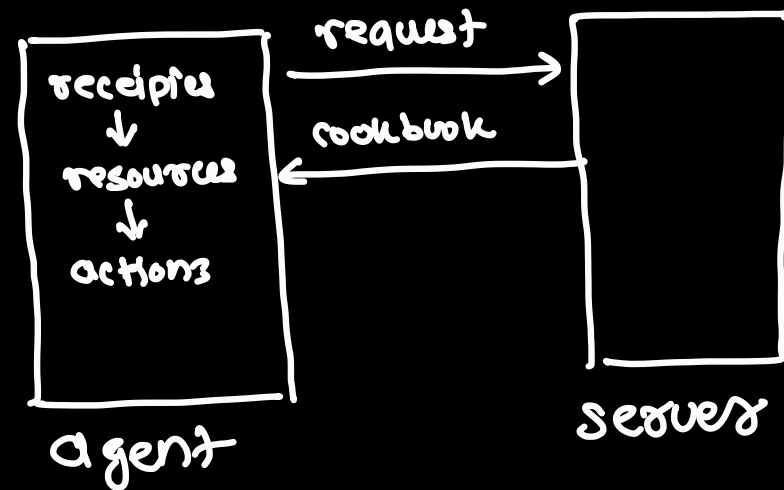


- chef-zero is a very lightweight Chef Infra Server that runs in-memory on the local machine
- This allows the Chef Infra Client to be run against the chef-repo as if it were running against the Chef Infra Server
- chef-zero was originally a standalone tool; it is enabled from within the Chef Infra Client by using the --local-mode option
- chef-zero is very useful for quickly testing and validating the behavior of the Chef Infra Client, cookbooks, recipes, and run-lists before uploading that data to the actual Chef Infra Server

Node Convergence



- Chef Infra Client configures the system based on the information that has been collected
- Each resource is executed in the order identified by the run-list, and then by the order in which each resource is listed in each recipe
- Each resource in the resource collection is mapped to a provider
- The provider examines the node, and then does the steps necessary to complete the action
- And then the next resource is processed
- Each action configures a specific part of the system
- This process is also referred to as convergence
- This is also referred to as the "execution phase"





Cookbook

Cookbooks



- A cookbook is the fundamental unit of configuration and policy distribution
- A cookbook defines a scenario and contains everything that is required to support that scenario:
 - Recipes that specify the resources to use and the order in which they are to be applied
 - Attribute values
 - File distributions
 - Templates
 - Extensions to Chef, such as custom resources and libraries
- Chef Infra Client uses Ruby as its reference language for creating cookbooks and defining recipes, with an extended DSL for specific resources
- Chef Infra Client will run a recipe only when asked
- When Chef Infra Client runs the same recipe more than once, the results will be the same system state each time
- When a recipe is run against a system, but nothing has changed on either the system or in the recipe, Chef Infra Client won't change anything

Cookbook Components



■ Recipes

- All recipes reside in recipes/ directory
- A recipe is the most fundamental configuration element within the organization
- A recipe
 - Is authored using Ruby, which is a programming language designed to read and behave in a predictable manner
 - Must define everything that is required to configure part of a system
 - Must be stored in a cookbook
 - May be included in another recipe
 - May have a dependency on one (or more) recipes
 - Must be added to a run-list before it can be used by Chef Infra Client
 - Is always executed in the same order as listed in a run-list

↳ list of recipes to execute

Cookbook Components



■ Attributes

- All attributes reside in recipes/ directory
- An attribute can be defined in a cookbook and then used to override the default settings on a node
- When a cookbook is loaded during a Chef Infra Client run, these attributes are compared to the attributes that are already present on the node
- Attributes that are defined in attribute files are first loaded according to cookbook order
- For each cookbook, attributes in the default.rb file are loaded first, and then additional attribute files (if present) are loaded in lexical sort order
- When the cookbook attributes take precedence over the default attributes, Chef Infra Client applies those new settings and values during a Chef Infra Client run on the node.

Cookbook Components



■ Files

- All static files reside in files/ directory
- A file distribution is a specific type of resource that tells a cookbook how to distribute files, including by node, by platform, or by file version

■ Libraries

- All libraries reside in libraries/ directory
- A library allows the use of arbitrary Ruby code in a cookbook, either as a way to extend the Chef Infra Client language or to implement a new class

■ Custom resources

- All custom resources reside in resources/ directory
- A custom resource is an abstract approach for defining a set of actions and (for each action) a set of properties and validation parameters

■ Templates

- All templates reside in templates/ directory
- A template is a file written in markup language that uses Ruby statements to solve complex configuration scenarios



Ohai

Chef Attribute



- An attribute is a specific detail about a node
- Attributes are used by Chef Infra Client to understand:
 - The current state of the node
 - What the state of the node was at the end of the previous Chef Infra Client run
 - What the state of the node should be at the end of the current Chef Infra Client run
- Attributes are defined by:
 - The state of the node itself
 - Attributes passed via JSON on the CLI
 - Cookbooks (in attribute files and/or recipes)
 - Roles
 - Environments
 - Policyfiles

Attribute types



- **Automatic**

- An automatic attribute contains data that is identified by Ohai at the beginning of every Chef Infra Client run
- An automatic attribute cannot be modified and always has the highest attribute precedence

- **Default**

- A default attribute is automatically reset at the start of every Chef Infra Client run and has the lowest attribute precedence
- Use default attributes as often as possible in cookbooks



- Ohai is a tool that is used to collect system configuration data, which is provided to Chef Infra Client for use within cookbooks
- Ohai is run by Chef Infra Client at the beginning of every Chef run to determine system state
- Ohai includes many built-in plugins to detect common configuration details as well as a plugin model for writing custom plugins
- The types of attributes Ohai collects include but are not limited to:
 - Operating System
 - Network
 - Memory
 - Disk
 - CPU
 - Kernel
 - Host names
 - Virtualization



- Attributes that are collected by Ohai are automatic level attributes, in that these attributes are used by Chef Infra Client to ensure that these attributes remain unchanged after Chef Infra Client is done configuring the node
- Ohai collects data for many platforms, including AIX, Darwin, Linux, FreeBSD, OpenBSD, NetBSD, Solaris, and any Microsoft Windows operating systems
- E.g.
 - `node['platform']`
 - `node['platform_version']`
 - `node['ipaddress']`
 - `node['macaddress']`
 - `node['fqdn']`
 - `node['hostname']`
 - `node['domain']`



Templates



What is template?

- A cookbook template is an Embedded Ruby (ERB) template that is used to dynamically generate static text files
- Templates may contain Ruby expressions and statements, and are a great way to manage configuration files
- Use the **template** resource to add cookbook templates to recipes
- Place the corresponding Embedded Ruby (ERB) template file in a cookbook's /templates directory

```
template '/var/www/html/index.html' do
  source 'index.html.erb'
  owner 'root'
  group 'root'
end
```

Variables



- An Embedded Ruby (ERB) template allows Ruby code to be embedded inside a text file within specially formatted tags
- Ruby code can be embedded using expressions and statements
- An expression is delimited by `<%=` and `%>`
- For example:

```
<html>
  <head>
    <title>Index</title>
  </head>
  <body>
    <h2>Hostname: <%= node["hostname"]%> </h2>
    <h2>IP address: <%= node["ipaddress"]%> </h2>
  </body>
</html>
```